

Marketplace Aggregator — Approach + Prototype

Variant 2 of 3 · Short write-up plus a minimal working app on AWS

Estimated time: 1 day (≈6–8 hours)

Deliverables: (1) a short written approach document, (2) a minimal working app deployed to AWS with a mocked marketplace integration.

Context

We are building a unified wrapper around third-party marketplaces (eBay, Facebook Marketplace, and eventually others) that lets a seller list a product once, push it to one or more marketplaces, and have every subsequent event — a sale, a buyer comment, a question, a price-change request — flow back into a single aggregated view where the seller can act.

We want to see both **how you think** and **how you build**. The written portion shows us your architectural instincts; the prototype shows us you can stand something up on AWS quickly and cleanly. The marketplace side can be fully mocked — we are not expecting a real eBay or Facebook integration in a day.

The problem

Design and build a minimal slice of the system:

- A simple web frontend where a user can create a product listing (title, description, price — photos optional).
- A backend endpoint that accepts the listing and "publishes" it to a **mocked** marketplace. The mock should behave like a realistic third party: async, rate-limited, occasionally failing.
- A way for the mocked marketplace to push events back — at minimum **item sold** and **new comment**. This can be a second endpoint you hit manually, a scheduled job, or a small script that fires webhooks at your system.
- A persistence layer that stores listings and an aggregated activity feed per listing.
- A UI view (can be extremely plain) that shows all listings and their most recent activity.

Pick one marketplace as the **conceptual** reference and justify the choice in the write-up, even though the integration itself is mocked.

You are free to use any approach — a classical API-integration layer, an agentic system, a hybrid, a message-driven pipeline, anything. We care that the reasoning is sound and that the solution is **safe** (credential handling, authZ, idempotency, abuse resistance) and **cost-effective** at both low and moderate volume.

*Deploy to AWS. **How you build, package and deploy the system is entirely your call** — framework, language, deployment tooling, data stores, everything. We are explicitly not prescribing any of it. The choices you make, and the reasons you make them, are part of what we are evaluating.*

Deliverable 1 — Approach document (1–2 pages)

A short Markdown document covering:

- **Approach summary.** One paragraph. API-driven, agentic, or hybrid — and why.
- **Architecture.** A small diagram plus the AWS services you chose and why.
- **Reference marketplace.** Which one, its auth model, rate limits, webhook story, known pitfalls.
- **Safety.** Credential storage, multi-tenant isolation, idempotency of publish, retry strategy.
- **Cost.** Rough monthly cost at 10 sellers / 1k listings / 10k events and where the first cost wall sits.
- **What you would cut, what you would build next.** Your prioritisation if this were a real product.

Keep it tight. Two sharp pages beats ten padded ones.

Deliverable 2 — Prototype

A working application deployed to your own AWS account.

Must-have

- Deployable to AWS with a single documented command from a clean clone.
- A frontend served via CloudFront (or equivalent) with a form to create a listing and a view to see listings plus their activity.
- A backend POST /listings endpoint that persists a listing and kicks off a publish flow.
- A **mock marketplace**. Can live in the same project as a separate module or as an external tiny service — either is fine, but the boundary between "our system" and "the marketplace" must be clean. The mock should accept a publish call and respond async, occasionally fail (10–20% synthetic error rate) to prove your retry / idempotency works, and emit events (item_sold, new_comment) back to your system via webhook.
- A webhook receiver that ingests marketplace events, validates them, and writes to the aggregated activity feed.
- Persistence. DynamoDB is fine; Postgres (Aurora Serverless / RDS) if you prefer, with justification.
- A README.md that tells us how to deploy it, how to tear it down, and roughly what it costs to leave running for a day.

Nice-to-have (do not stretch the day)

- Basic auth (Cognito, Clerk, Auth.js, a shared token — anything sensible).
- Event replay and dead-letter handling.
- One integration or smoke test that runs against the deployed stack.
- A short cost / observability note — which CloudWatch metrics or alarms you would add first.

Out of scope

- Real eBay / Facebook OAuth or API calls.
- Polished UI. Plain HTML / Tailwind is fine.
- Multi-region, multi-account, SSO.

Safety and cost expectations

We deliberately want to see **cost-aware** choices — this is a prototype, not a production cluster.

- Prefer pay-per-request / on-demand pricing (DynamoDB on-demand, Lambda, etc.) over provisioned capacity.
- Avoid anything that costs money 24/7 that you do not actually need (NAT gateways, idle ECS services, Aurora provisioned, etc.). If you use one, justify it in the README.
- Make the stack easy to destroy — a single teardown command should leave nothing behind that will charge us next month.

- Do not commit AWS keys, marketplace tokens, or any secret to the repo. Use AWS Secrets Manager or SSM Parameter Store and document it.

Evaluation criteria

Roughly in this order:

- **The prototype actually works** — we can deploy it, create a listing, see a mock sale come back, and watch it appear in the activity feed.
- **Code quality** — clear structure, sensible typing, a README we can follow, no dead code.
- **AWS depth** — right services for right reasons, clean project layout, reasonable IAM.
- **Tooling judgement** — your framework, deployment, and packaging choices are themselves part of the evaluation.
- **Safety** — idempotent publish, retries without duplicates, secrets handled properly, webhook signature verification (even on the mock).
- **Cost awareness** — visible in both the write-up and the implementation choices.
- **Product sense** — did you make any small decisions that make the seller's life easier.

Submission

- A **public or shared private Git repo** (GitHub / GitLab / Bitbucket).
- README.md at the root with prerequisites, deploy instructions, environment variables, how to trigger the mock events, how to tear it down, and estimated cost/day.
- Approach document as APPROACH.md in the repo, or linked from the README.
- A short Loom or screen recording (≤ 5 min) walking us through a listing → sale → activity-feed flow. Optional but appreciated.
- If the deployed URL is still up when you submit, include it.

If anything in the brief is ambiguous, make a judgement call and note it in the README. We treat "asked a sharp clarifying question" and "made a sensible assumption and flagged it" as equally good.